Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
**«САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
АЭРОКОСМИЧЕСКОГО ПРИБОРОСТРОЕНИЯ»**
(СПбГУАП)

Кафедра компьютерных технологий и программной инженерии

КУРСОВОЙ ПРОЕКТ

ПОЯСНИТЕЛЬНАЯ ЗАПИСКА К КУРСОВОМУ ПРОЕКТУ

по дисциплине
«ОБЪЕКТНО-ОРИЕНТИРОВАННОЕ ПРОГРАММИРОВАНИЕ»

Разработка настольного приложения для организации взаимодействия объектов библиотечной информационной системы

Руководитель проекта Е. О. Шумова
Работу выполнил студент группы М412
В. Д. Панков

Санкт-Петербург 2026

Задание на курсовой проект
по дисциплине
«Объектно-ориентированное программирование»

Студенту группы **М412 — В. Д. Панкову**.

Тема проекта: **«Разработка настольного приложения для организации взаимодействия объектов библиотечной информационной системы»**.

Исходные данные: предметная область «Библиотека», иерархия сущностей, графическое Desktop-приложение, локальное хранение в SQLite через Room 3 и применение шаблонов проектирования.

Проект должен содержать:
1. анализ предметной области и формулировку функционала
2. разработку иерархии классов сущностей, управляющих и интерфейсных классов
3. обоснование шаблонов проектирования и построение диаграмм
4. разработку Desktop-приложения и тестовых сценариев
5. тестирование с фиксацией фактических результатов
6. оформление пояснительной записки и презентации

Срок сдачи законченного проекта: не указан в доступных материалах проекта.
Руководитель проекта: Е. О. Шумова.
Дата выдачи задания: не указана в доступных материалах проекта.
Формальное согласование темы: в доступных материалах проекта не зафиксировано.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ 4

1. ПОСТАНОВКА ЗАДАЧИ

. 5

1.1. Анализ предметной области 5

1.2. Формулировка функционала 6

2. РАЗРАБОТКА КЛАССОВ

. 6

2.1. Классы сущностей 6

2.2. Управляющие классы 8

2.3. Интерфейсные классы 8

2.4. Используемые паттерны проектирования 9

2.4.1. Repository 9

2.4.2. MVVM 10

2.4.3. Strategy 10

2.4.4. Factory 11

2.4.5. Observer 12

2.4.6. State 13

2.4.7. Facade 14

2.5. Общая архитектура и схема хранения 15

3. РАЗРАБОТКА ПРИЛОЖЕНИЯ

. 17

3.1. Разработка интерфейса приложения 17

3.2. Реализация методов классов 19

3.3. Контрольный сценарий приложения 20

4. ТЕСТИРОВАНИЕ

. 21

ЗАКЛЮЧЕНИЕ 22
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ 23
А.1. Контракт сущности и генератор UUID 24
А.2. Состояние выдачи 24
А.3. Политика выдачи 25
А.4. Транзакции DAO 25
Б.1. JVM-точка входа 26
Б.2. Наблюдаемое состояние ViewModel 26
Б.3. Проверка персистентности и идемпотентного возврата 26
Б.4. Проверка M:N-связи 27
Б.5. Идемпотентный seed 27

ВВЕДЕНИЕ

Объектно-ориентированное программирование позволяет представить предметную область как систему взаимодействующих объектов с ясными ответственностями. В библиотечной информационной системе такими объектами являются читатели, сотрудники, издания, филиалы и факты выдачи. Для завершённой реализации важно не только описать классы, но и связать их с пользовательскими сценариями, хранением данных и проверяемыми инвариантами.

Цель курсового проекта – разработать и протестировать настольное графическое приложение, демонстрирующее взаимодействие иерархии классов библиотечной предметной области с применением принципов ООП и шаблонов проектирования. Финальная реализация представляет собой приложение Desktop JVM на Kotlin и Compose Desktop. Сохранение выполняется в локальной SQLite-базе с использованием Room 3 и `BundledSQLiteDriver`.

Для достижения цели выполнены следующие задачи:

— проанализирована предметная область и выделены сущности;

— построены иерархии `Person` → `Reader`/`Librarian` и `Publication` → `Book`/`Periodical`;

— реализована многозначная связь `Reader` ↔ `Publication` через ассоциативную сущность `Loan`;

— разделены сущностные, управляющие и интерфейсные классы;

— применены семь паттернов: Repository, MVVM, Strategy, Factory, Observer, State и Facade;

— обеспечены непрозрачные UUID, постоянное хранение, транзакционные выдача и возврат, а также идемпотентная загрузка демонстрационных данных;

— выполнены JVM unit- и integration-тесты, а результат сборки сохранён в артефактах.

В разделе 1 рассматриваются предметная область и функции системы. Раздел 2 содержит разработку классов, семь отдельных диаграмм паттернов и общую архитектурную диаграмму. В разделе 3 описаны интерфейс, методы и контрольный сценарий. Раздел 4 фиксирует фактический результат тестирования. В приложениях приведены только репрезентативные фрагменты финального кода и тестов.

1. ПОСТАНОВКА ЗАДАЧИ

1.1. Анализ предметной области

Проект моделирует обслуживание читателей в библиотеке. Основные понятия предметной области и их роли следующие:

- **читатель** – зарегистрированный пользователь, которому могут выдаваться издания;
- **библиотекарь** – сотрудник, оформляющий выдачу и связанный с филиалом;
- **издание** – единица фонда; в проекте представлены книга и периодическое издание;
- **филиал** – место хранения фонда;
- **выдача** – зафиксированный факт передачи издания читателю с датами и состоянием.

Существительные предметной области преобразуются в сущностные классы, а глаголы – в операции приложения: зарегистрировать читателя, добавить издание, сохранить, выдать, вернуть, просмотреть данные и загрузить демонстрационный набор. Идентификаторы сущностей представлены строками, но их значения являются непрозрачными: `UuidIdGenerator` создаёт UUID внутри приложения, а пользователь выбирает читателя и издание по предметным признакам.

Многозначная зависимость является центральной связью модели. Один `Reader` может иметь много `Loan`, а одно `Publication` может фигурировать во многих `Loan`, если доступны разные экземпляры или выдачи относятся к разным моментам времени. Каждая выдача ссылается на одного читателя, одно издание и одного библиотекаря.

В базе эта связь выражена внешними ключами `readerId`, `publicationId` и `librarianId` таблицы `loans`.

Приложение ограничено настольным JVM-сценарием. В `core` и `app` объявлена цель `jvm()`, точка входа создаёт окно Compose Desktop, а зависимости распределены между доменным ядром и графическим модулем.

1.2. Формулировка функционала

Функциональность финальной версии определяется исходным кодом и тестами:

Группа	Реализованные возможности
Читатели	создание, поиск по ФИО, отображение и удаление читателей; валидация полей; сохранение через репозиторий
Издания	создание, поиск по названию или автору, отображение и удаление книг/периодики; выбор филиала и контроль экземпляров
Выдачи	выбор читателя и издания; проверка библиотекаря, политики и доступности; атомарное оформление; отображение выдач и возврат
Целостность	непрозрачные UUID; внешние ключи; запрет удаления читателя или издания с активной выдачей; возврат только один раз
Администрирование	явная транзакционная загрузка связанного демонстрационного набора; повторный вызов не создаёт дубликаты; безопасная очистка всех таблиц
Хранение	запись в SQLite, закрытие и повторное открытие Room-базы с восстановлением доменных объектов

Интерфейс предоставляет четыре вкладки: Readers, Publications, Loans и Architecture. На первых двух доступны создание, человекочитаемый поиск, просмотр и удаление; на третьей выбираются читатель и доступное издание, после чего выполняются Issue и Return. Четвёртая вкладка показывает ООП-иерархии, M:N-связь, слои и семь паттернов, а также содержит явные команды seed и транзакционной очистки. Технические идентификаторы в поля формы не выводятся.

2. РАЗРАБОТКА КЛАССОВ

2.1. Классы сущностей

Корневой контракт `LibraryEntity` содержит только непрозрачный `id`. Через него образованы две иерархии: `Person` с реализациями `Reader` и `Librarian`, а также `Publication`

с реализациями `Book` и `Periodical`. `LibraryBranch` и `Loan` также реализуют `LibraryEntity`, но не являются подтипами `Person` или `Publication`.

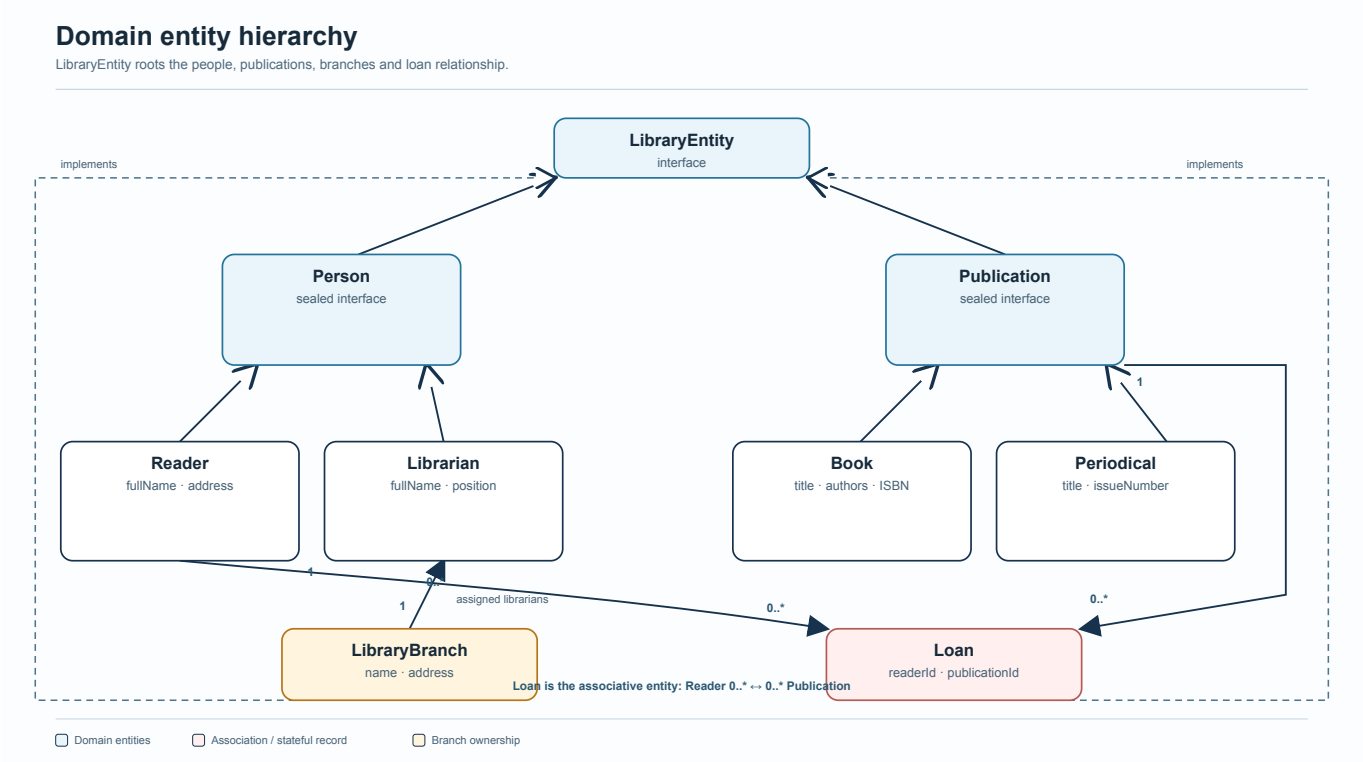


Рисунок 1 — Иерархия классов сущностей

Класс	Категория	Основные поля	Ответственность
LibraryEntity	интерфейс	`id: String`	единый контракт идентификации
Person	интерфейс	`fullName: String`	общий контракт человека
Reader	сущность	`id`, `fullName`, `birthYear`, `address`, `organization`	зарегистрированный читатель и проверка данных
Librarian	сущность	`id`, `fullName`, `position`, `branchId`	сотрудник филиала, оформляющий выдачу
Publication	интерфейс	`title`, `authors`, `publisher`, `publicationYear`, `totalCopies`, `availableCopies`, `branchId`	общий контракт фонда
Book	сущность	общие поля `Publication` и необязательный `isbn`	книга с инвариантами фонда
Periodical	сущность	общие поля `Publication` и `issueNumber`	выпуск периодического издания
LibraryBranch	сущность	`id`, `name`, `address`	место хранения фонда
Loan	сущность	`readerId`, `publicationId`, `librarianId`, `issuedAt`, `dueAt`, `state`	ассоциативная связь и жизненный цикл выдачи

В LoanState определены состояния Active, Overdue и Returned(returnedAt). Переход returnAt переводит активную или просроченную выдачу в Returned; повторный возврат завершается доменной ошибкой. LoanPolicy задаёт проверку допустимости выдачи и срок возврата: в DefaultLoanPolicy срок зависит от типа публикации, а для читателя действует предел активных выдач.

2.2. Управляющие классы

Управляющий слой координирует сущности, хранилище и представление.

— LibraryService – доменный интерфейс фасада с операциями выдачи, возврата, защищённого удаления, repeat-safe seed и транзакционной очистки. RoomLibraryService зависит от доменного LibraryRepositoryBundle, проверяет связанные объекты, вызывает LoanPolicy и передаёт атомарные операции адаптерам.

— RoomRepositoryBundle реализует доменный LibraryRepositoryBundle и объединяет Room-адаптеры читателей, сотрудников, филиалов, изданий и выдач. Интерфейсы возвращают только доменные объекты и Flow; типы Room Record не выходят из persistence-слоя.

— LibraryEntityFactory создаёт валидные Reader, Librarian, LibraryBranch, Book, Periodical и Loan, скрывая генерацию UUID и проверку конструкторов.

— ReadersViewModel, PublicationsViewModel, LoansViewModel и AdministrationViewModel принимают события интерфейса, формируют UiState, выполняют поиск/валидацию и запускают корутины для операций.

— ApplicationContainer является явной точкой сборки зависимостей: создаёт базу, набор репозиториев и RoomLibraryService, а при закрытии освобождает базу.

2.3. Интерфейсные классы

Интерфейсный слой реализован composable-компонентами модуля app. main.kt создаёт ApplicationContainer, открывает Window и передаёт контейнер в App; загрузка демонстрационных данных выполняется только явной кнопкой. Компонент App создаёт

ViewModel и передаёт их в `LibraryScreen`. Внутри экрана находятся панели `ReadersPane`, `PublicationsPane`, `LoansPane` и `ArchitecturePane`.

Компоненты отвечают за компоновку элементов и пользовательские события, но не выполняют SQL и не меняют инвентарь напрямую. Состояния загрузки, пустого списка и ошибки отображаются через `CollectionUiState`. Список выдач получает имена читателей и названия публикаций, сопоставляя их с идентификаторами, которые остаются внутренними для приложения.

2.4. Используемые паттерны проектирования

Ниже каждый из семи паттернов рассмотрен отдельно. Диаграммы подключены из соответствующих финальных SVG-файлов каталога `docs/images`.

2.4.1. Repository

Repository отделяет доменную часть от конкретного хранилища. Интерфейсы `ReaderRepository`, `PublicationRepository` и `LoanRepository` описывают операции над доменными объектами, а классы с префиксом `Room` отображают записи Room и делегируют запросы `LibraryDao`. Благодаря этому ViewModel и сервис не знают SQL-представление таблиц.

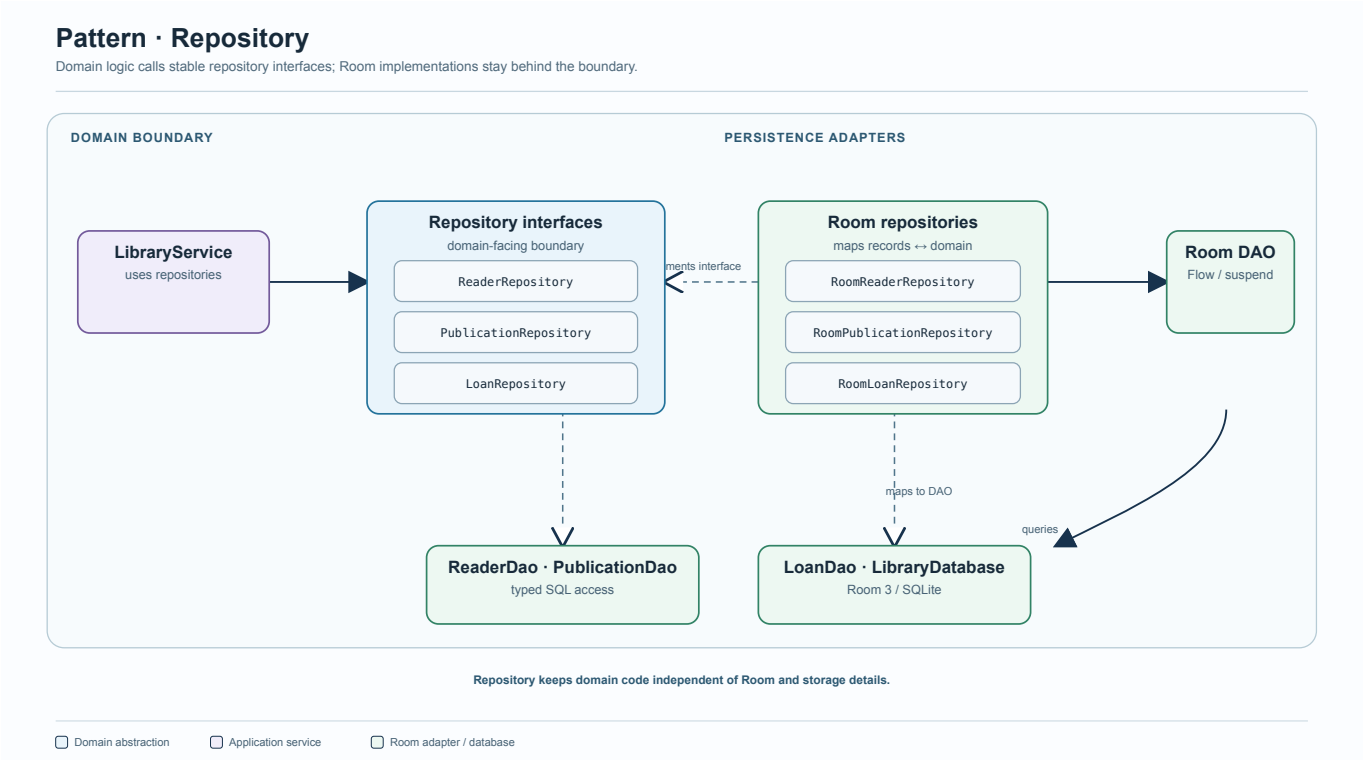


Рисунок 2 — Паттерн Repository: доменные интерфейсы и Room-адаптеры

2.4.2. MVVM

MVVM разделяет Compose-представление, состояние и сценарии. `LibraryScreen` и панели отображают `CollectionUiState`, ViewModel принимает действия пользователя, а `RoomLibraryService` и репозитории выполняют операции. ViewModel можно проверять отдельно от окна, что подтверждено тестом валидации читателя.

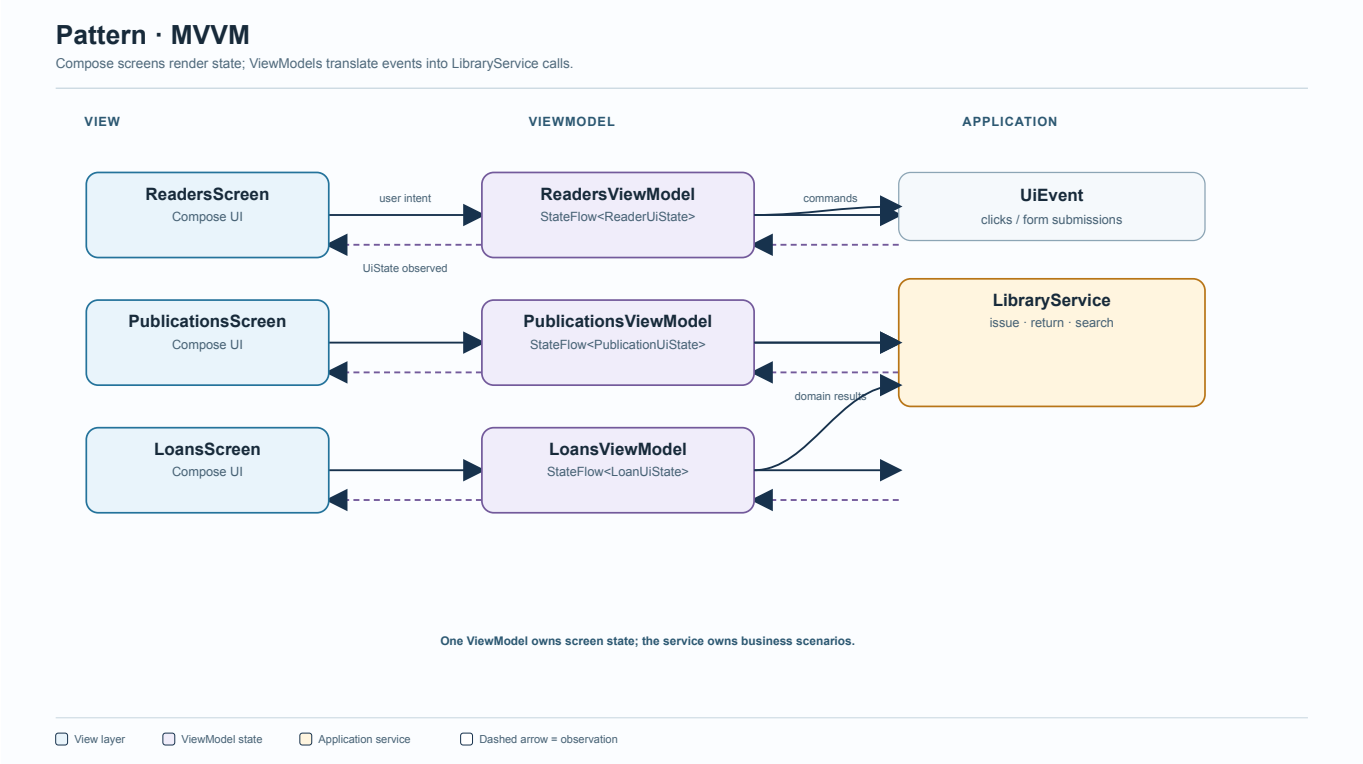


Рисунок 3 — Паттерн MVVM: Compose, ViewModel и application layer

2.4.3. Strategy

Strategy представлен интерфейсом `LoanPolicy`. `RoomLibraryService` получает политику через конструктор и использует её для решения `canIssue` и вычисления `dueAt`. `DefaultLoanPolicy` применяет разные сроки для `Book` и `Periodical`, поэтому правило выдачи можно заменить без изменения фасада и хранилища.

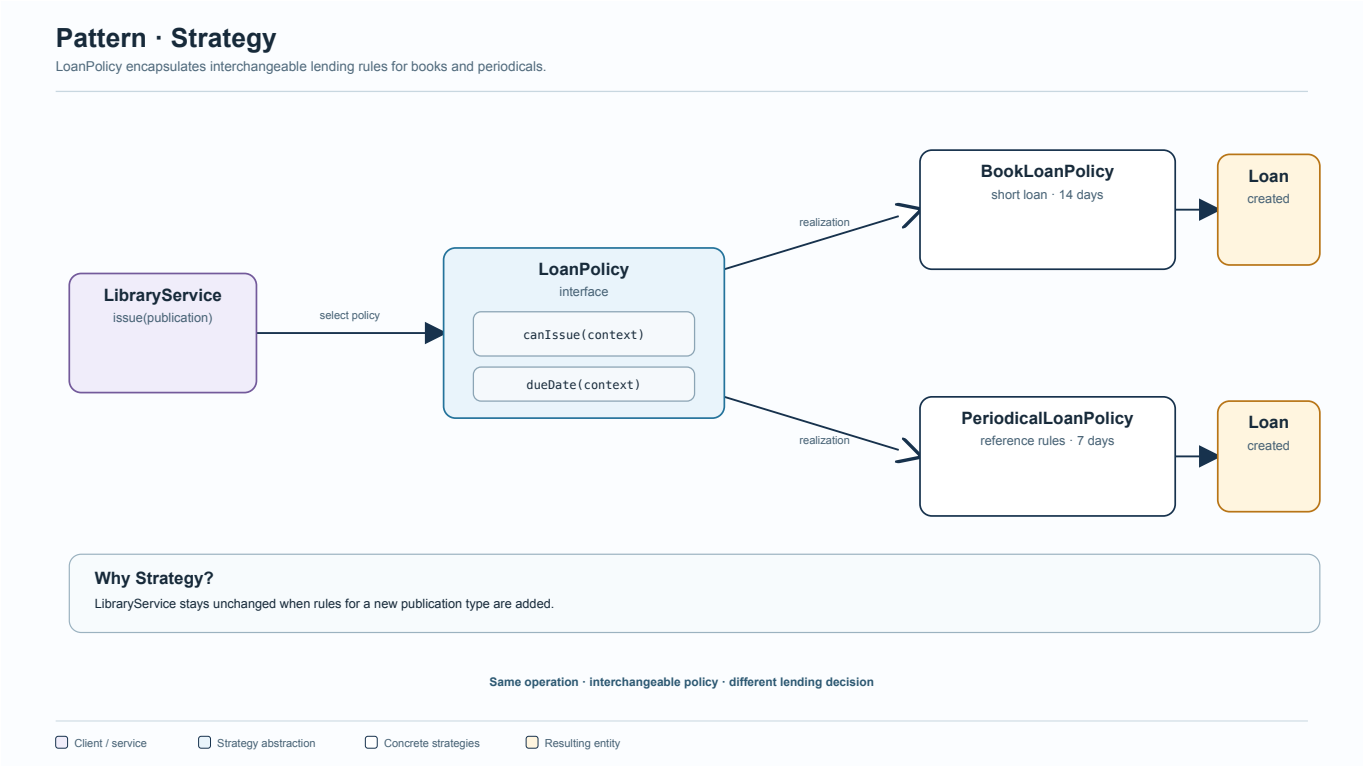


Рисунок 4 — Паттерн Strategy: LoanPolicy и правила выдачи

2.4.4. Factory

`LibraryEntityFactory` инкапсулирует создание валидных сущностей и получение UUID через `IdGenerator`. В `seedDemo` фабрика используется для построения филиала, библиотекаря, двух читателей, книги, периодики и выдачи. Затем записи передаются в одну транзакцию Room. Клиенту не требуется знать порядок конструирования объектов.

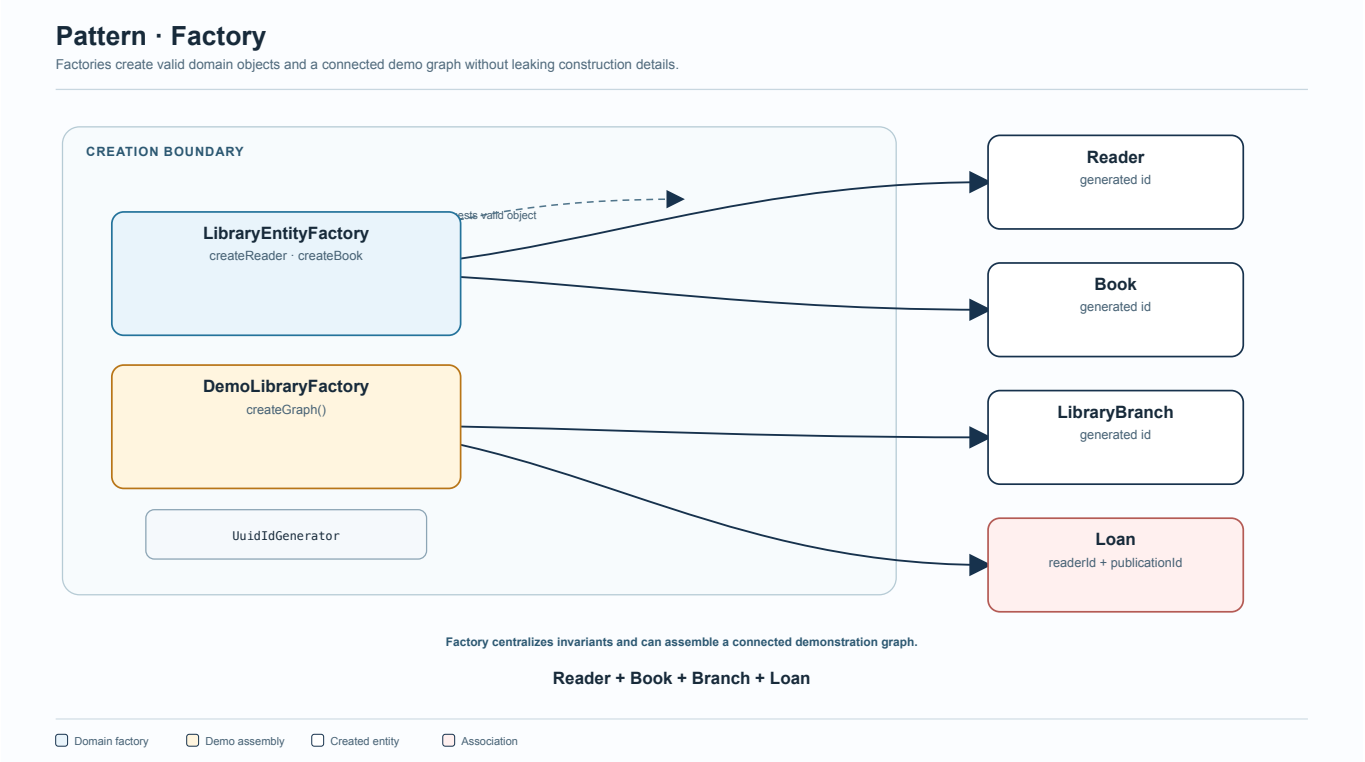


Рисунок 5 — Паттерн Factory: создание сущностей и демонстрационного графа

2.4.5. Observer

В роли источника изменений выступают `Flow<List<...>>`, которые возвращают DAO. Репозитории преобразуют записи в доменные объекты, а ViewModel переводит поток в `StateFlow<CollectionUiState<...>>` через `stateIn`. Compose получает состояние через `collectAsState`. Изменение записи автоматически распространяется на наблюдающий экран.

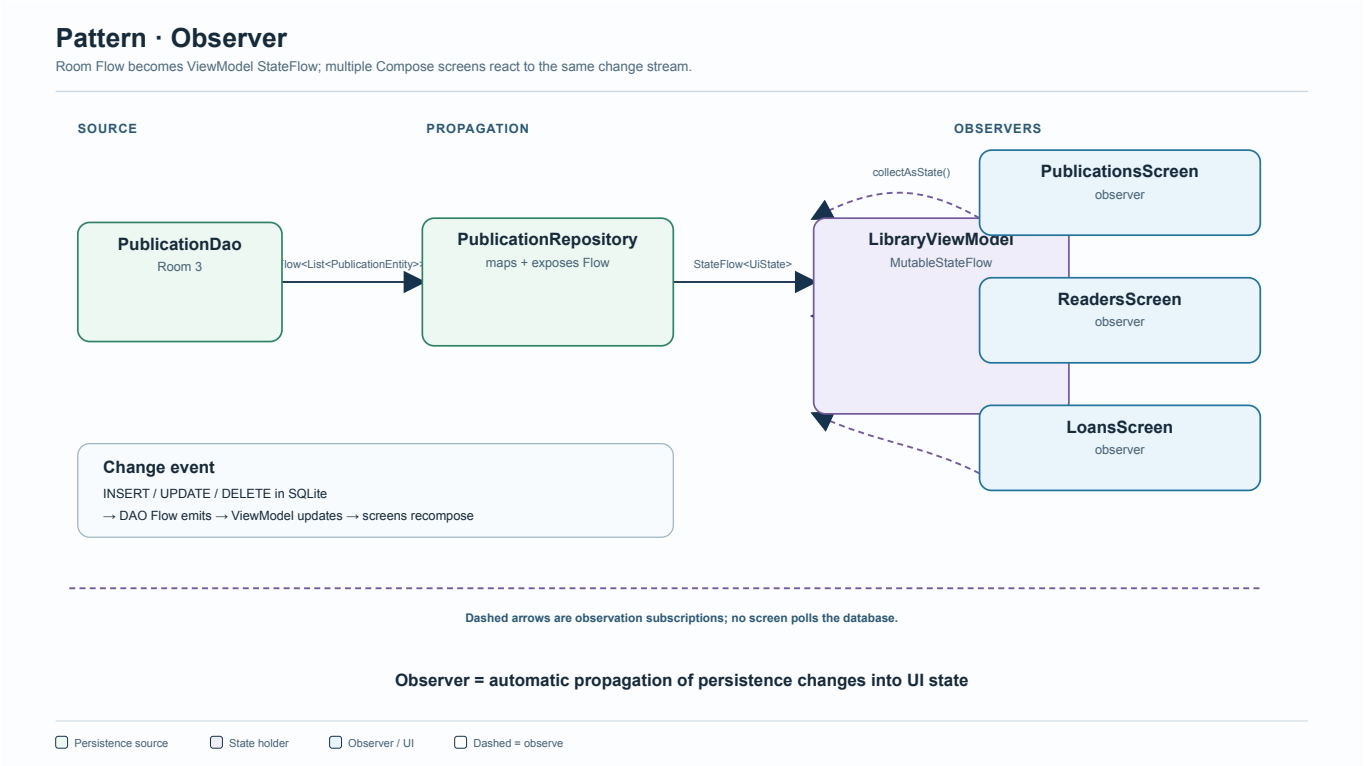


Рисунок 6 — Паттерн Observer: Room Flow → StateFlow → Compose

2.4.6. State

`Loan` хранит объект состояния `LoanState`. Состояния `Active`, `Overdue` и `Returned` выражают жизненный цикл выдачи, а функция `returnAt` разрешает переход к возврату только из первых двух состояний. При повторном вызове возврата DAO также возвращает уже завершённую запись, не увеличивая инвентарь второй раз.

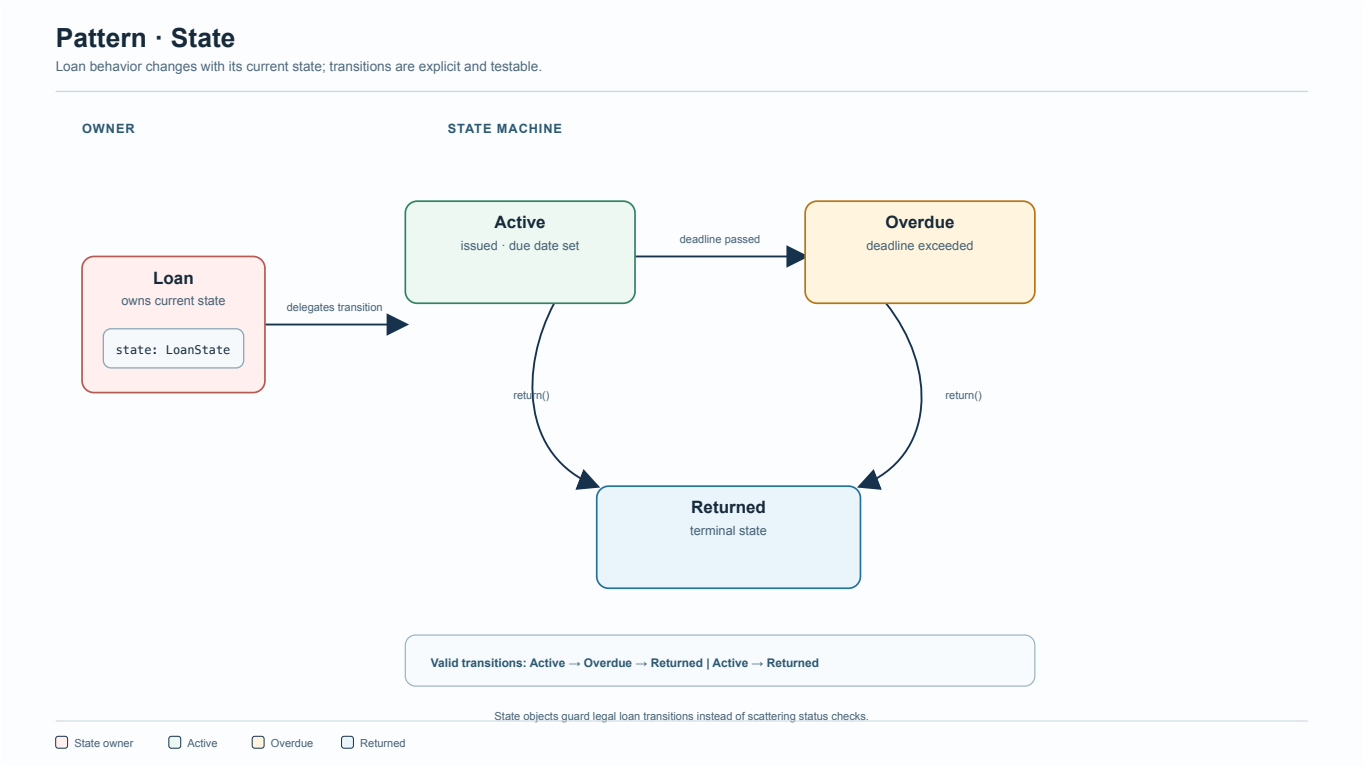


Рисунок 7 — Паттерн State: состояния и переходы Loan

2.4.7. Facade

`LibraryService` предоставляет клиенту короткие сценарии выдачи и возврата. `RoomLibraryService` скрывает поиск читателя, публикации и библиотекаря, применение `LoanPolicy`, генерацию идентификатора и вызов транзакционного репозитория. Тем самым ViewModel не координирует несколько DAO самостоятельно.

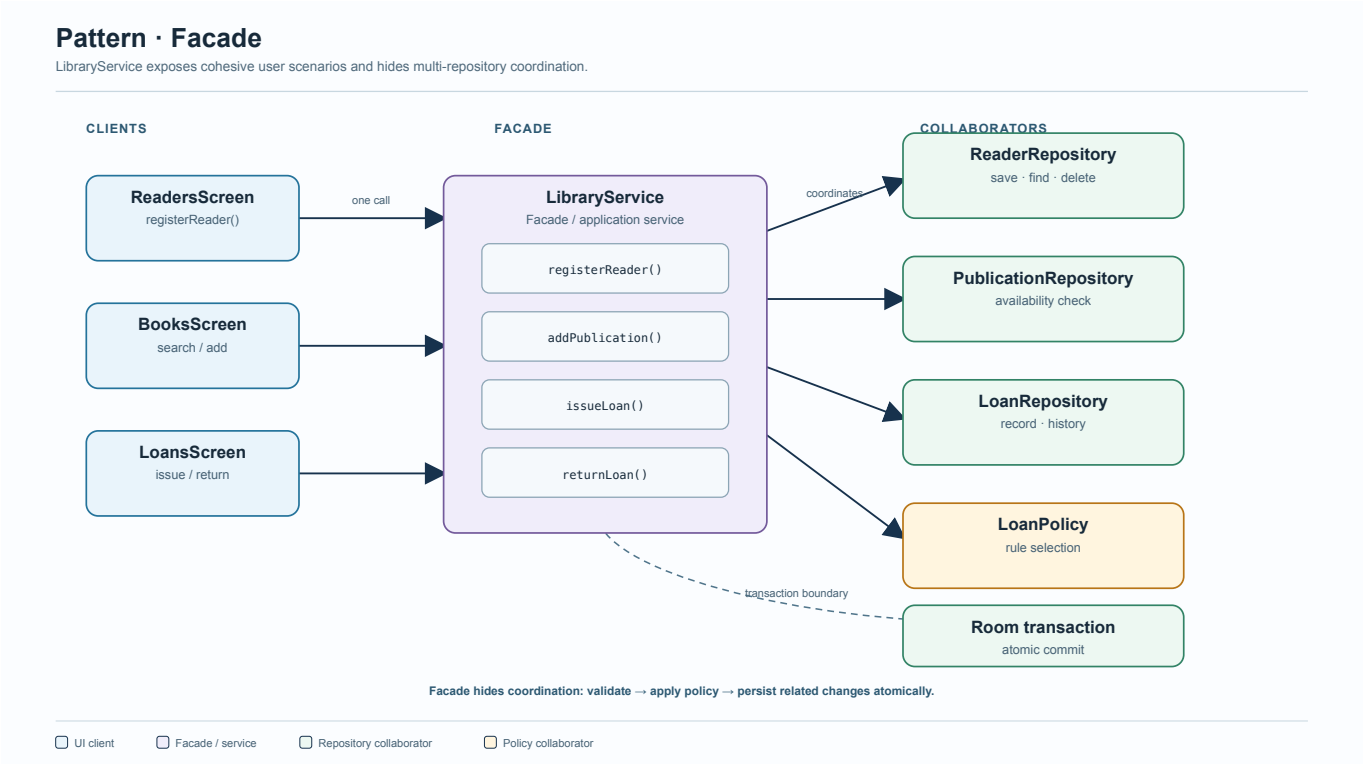


Рисунок 8 — Паттерн Facade: LibraryService и координация сценария

2.5. Общая архитектура и схема хранения

Архитектура разделена на пять логических уровней. Доменный слой содержит сущности, состояния, политики и фабрику. Persistence-слой содержит записи Room, DAO, базу и преобразования. Репозитории образуют границу между этими слоями. Application-слой реализует фасад и сценарии. UI-слой содержит Compose Desktop, ViewModel и контейнер зависимостей.

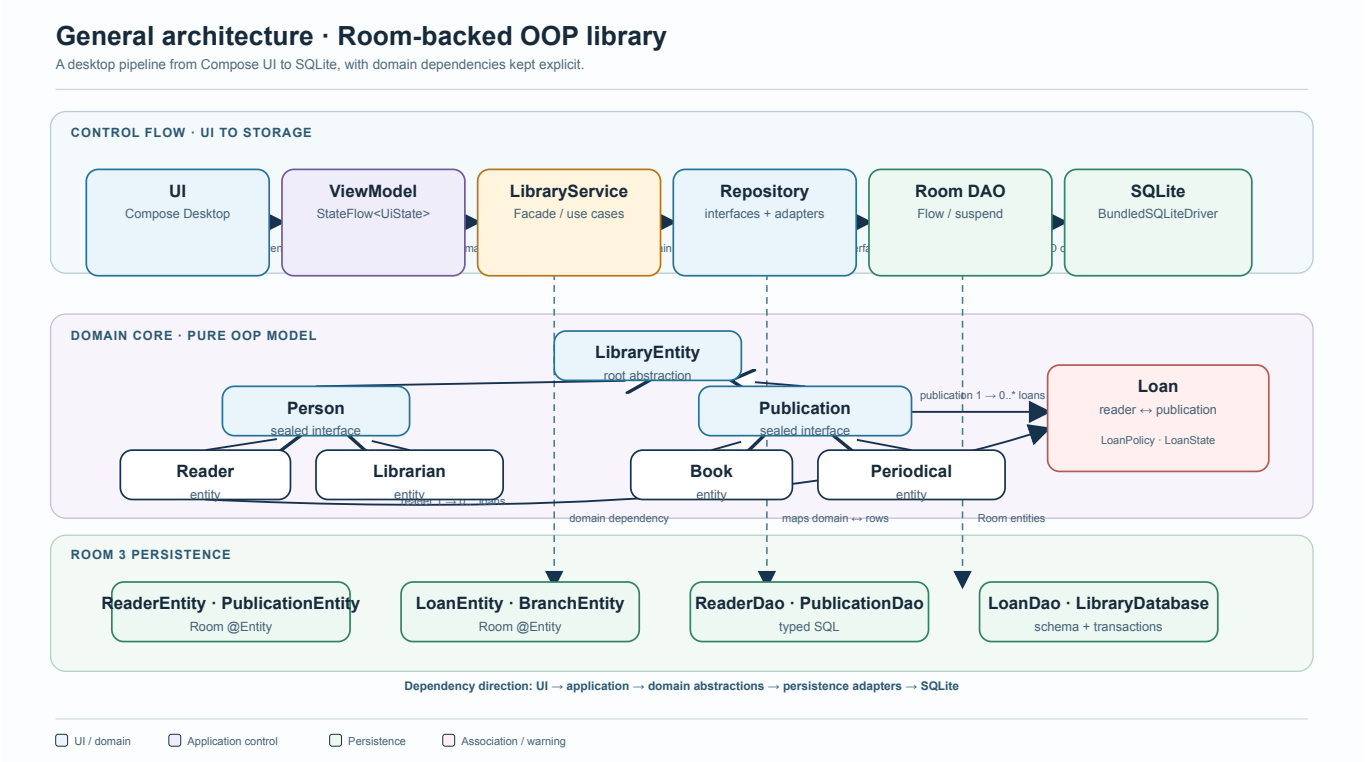


Рисунок 9 — Общая архитектурная диаграмма финального приложения

`LibraryDatabase` объявляет пять таблиц: `readers`, `librarians`, `branches`, `publications` и `loans`. Таблица `publications` хранит дискриминатор `kind`, библиографические поля и количества экземпляров. Для `loans` определены внешние ключи на читателя, публикацию и библиотекаря, а также индексы по этим полям.

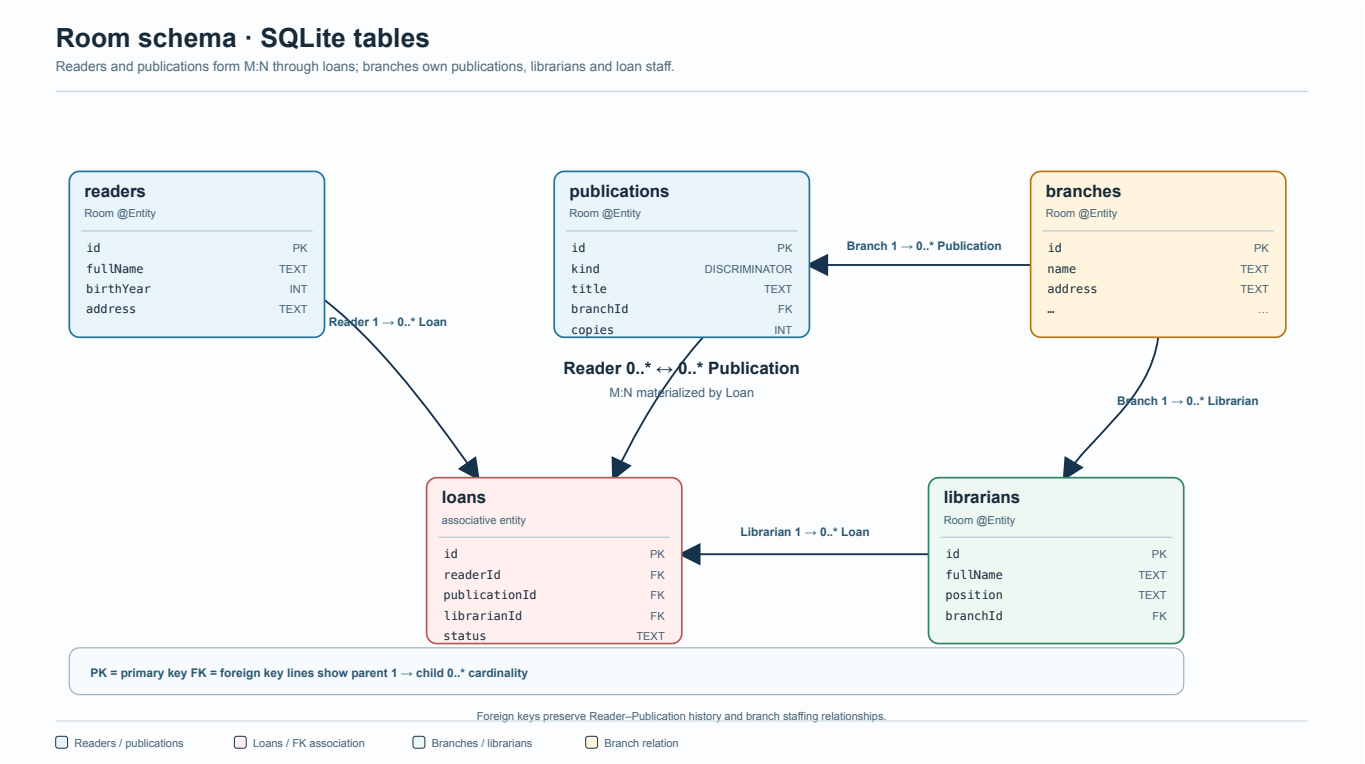


Рисунок 10 — Схема Room/SQLite и внешние ключи

Выдача выполняется внутри `@Transaction`: сначала уменьшается `availableCopies` условным SQL-запросом, затем вставляется `LoanRecord`; при отсутствии экземпляра транзакция возвращает отказ. Возврат сначала проверяет текущую запись и помечает её как `RETURNED`, затем увеличивает доступное количество ровно один раз. Загрузка демонстрационных данных также транзакционна: если выдача невозможна, записи подготовленного набора откатываются. Проверка непустой таблицы филиалов делает повторный вызов `seedDemo` идемпотентным.

3. РАЗРАБОТКА ПРИЛОЖЕНИЯ

3.1. Разработка интерфейса приложения

Структура меню финального приложения соответствует четырём вкладкам главного экрана.

Вкладка	Компонент	Назначение
Readers	`ReadersPane`	поиск по ФИО, ввод полей, Add reader, список, Delete и состояния ошибки/загрузки

Вкладка	Компонент	Назначение
Publications	`PublicationsPane`	поиск по названию или автору, выбор филиала, добавление Book/Periodical, доступность и Delete
Loans	`LoansPane`	выбор читателя, выбор публикации с доступным экземпляром, Issue, список выдач и Return
Architecture	`ArchitecturePane`	иерархии, M:N, pipeline, семь паттернов, Seed demo data и Clear all data

Пользователь не вводит `id`, `readerId`, `publicationId` или `librarianId`. В панели выдач выбираются объекты по имени и названию, после чего ViewModel передаёт внутренние UUID сервису. Для публикаций доступны поля ISBN книги и номера выпуска периодики; состав объекта определяется нажатием соответствующей кнопки.

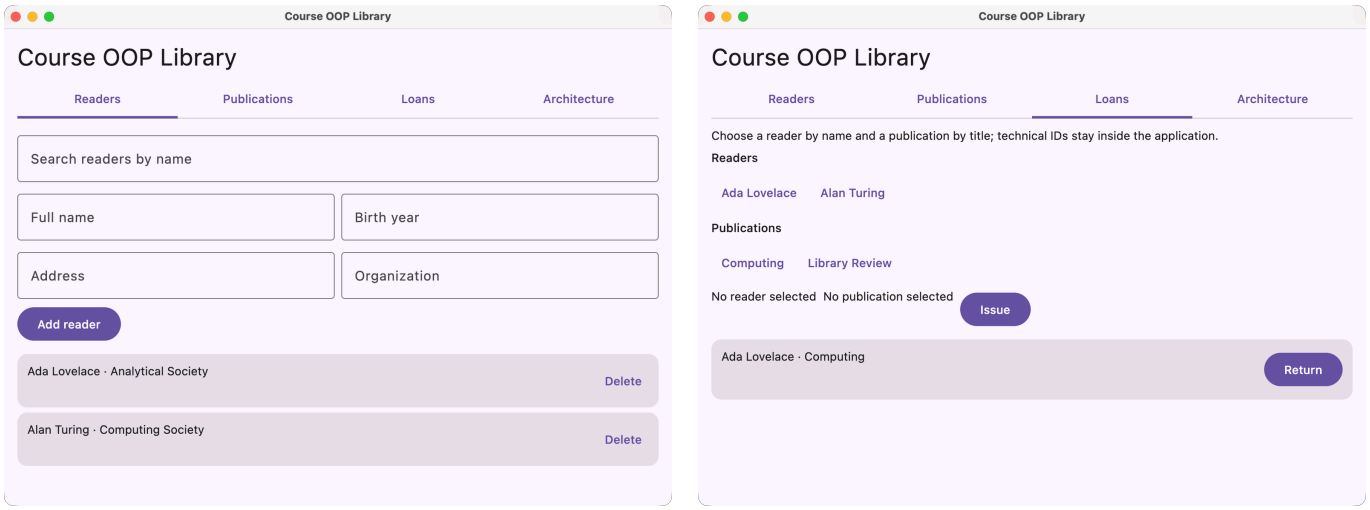


Рисунок 11 — Финальный интерфейс: поиск/управление читателями и сценарий выдачи

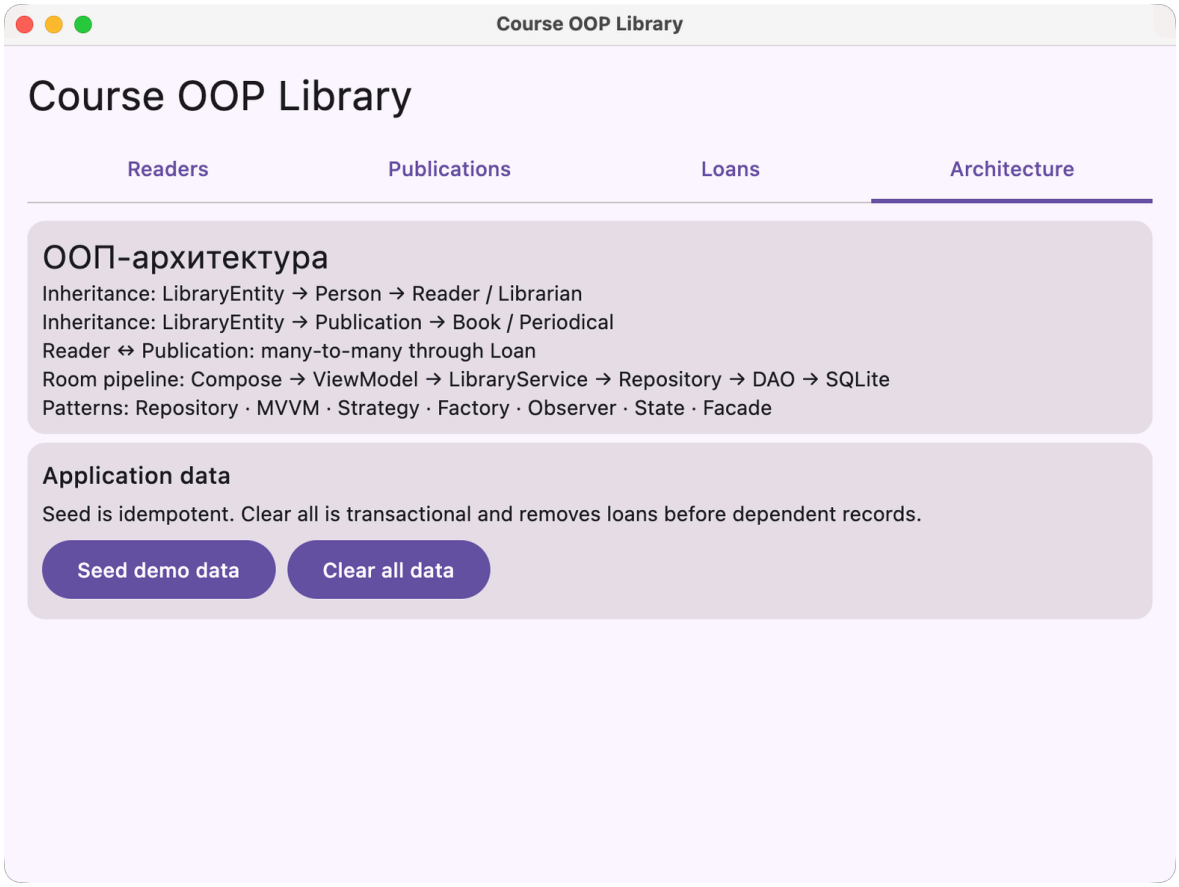


Рисунок 12 — Вкладка «ООП-архитектура» и явные операции демонстрационных данных

3.2. Реализация методов классов

Методы распределены по ответственностям. Конструкторы доменных классов проверяют непустые строки, допустимые годы, положительное общее количество и диапазон `availableCopies`. ViewModel проверяет преобразование текстовых полей до запуска фоновой операции. Репозитории выполняют отображение `Record` ↔ доменный объект.

Ключевой метод `issueLoan` в `RoomLibraryService` работает так:

1. получает читателя, публикацию и библиотекаря по внутренним идентификаторам;
2. передаёт читателя, публикацию и число активных выдач в `LoanPolicy`;
3. создаёт `Loan` с UUID, временем выдачи и рассчитанным сроком;
4. вызывает атомарную выдачу, которая уменьшает остаток и вставляет запись;
5. сообщает об ошибке, если подходящей копии нет.

Метод `returnLoan` делегирует `returnLoanAtomically`. DAO получает текущую запись, не меняет уже возвращённую выдачу, а для активной записи обновляет статус и увеличивает остаток. Поэтому повторный вызов идемпотентен.

Фрагмент доменного перехода состояния:

```
fun LoanState.returnAt(timestamp: Long): LoanState = when (this) {
    LoanState.Active, LoanState.Overdue → LoanState.Returned(timestamp)
    is LoanState.Returned → error("Loan has already been returned")
}
```

Фрагмент транзакционной выдачи из `LibraryDao`:

```
@Transaction
suspend fun issueLoanAtomically(record: LoanRecord): Boolean {
    if (decrementAvailable(record.publicationId) ≠ 1) return false
    insertLoan(record)
    return true
}
```

3.3. Контрольный сценарий приложения

При запуске `main` создаёт `ApplicationContainer` и открывает окно `Course OOP Library`; скрытая автоматическая загрузка данных не выполняется. На вкладке Architecture пользователь нажимает `Seed demo data`: в базу транзакционно попадают филиал, библиотекарь, два читателя, книга, периодика и одна активная выдача. Повторное нажатие не создаёт дубликаты. Команда `Clear all data` в одной транзакции удаляет сначала выдачи, затем зависимые публикации, читателей, сотрудников и филиалы.

Пользовательский сценарий после открытия окна: загрузить демонстрационные данные; найти читателя по имени и публикацию по названию или автору; добавить новые объекты; в Loans выбрать читателя и доступное издание; выполнить Issue; убедиться, что доступное количество уменьшилось; выполнить Return и убедиться, что количество восстановилось. Попытка удалить объект со связанной выдачей показывает понятную ошибку; ошибочный ввод отклоняется до записи.

4. ТЕСТИРОВАНИЕ

Тестирование выполнено на JVM и включает доменные проверки, интеграцию с реальной Room/SQLite-базой, транзакционные инварианты, связь многие-ко-многим, поиск/удаление/очистку, контрольный сценарий и ViewModel. Фактический итог зафиксирован в `artifacts/test-results.txt`: 17 тестов, 0 failures, 0 errors и 0 skipped. Полная сборка зафиксирована в `artifacts/build-results.txt`.

Тестовый класс	Тестов	Проверяемое поведение
`DomainRewriteTest`	4	UUID-идентификаторы, валидация домена, переход LoanState и различие правил LoanPolicy
`RoomPersistenceTest`	2	запись, закрытие, повторное открытие и восстановление; атомарная выдача, возврат и повторный возврат
`RoomInvariantTest`	2	запрет удаления связанных объектов; откат полностью неуспешной транзакции seed
`ManyToManyTest`	1	две выдачи одного издания разным читателям и корректное уменьшение инвентаря
`ControlScenarioTest`	1	успешный seed, идемпотентный повторный seed и ожидаемый граф данных
`RoomAdministrationTest`	2	защищённое удаление, доменная граница seed и транзакционная очистка с подсчётом удалённых записей
`ViewModelValidationTest`	1	отклонение некорректной формы без записи в репозиторий
`ViewModelOperationsTest`	4	поиск читателей/изданий, удаление и отображение ошибок сервисных операций

Основной сохранённый командный прогон:

```
./gradlew clean test build --no-daemon
Result: BUILD SUCCESSFUL
17 tests, 0 failures, 0 errors, 0 skipped
```

В `artifacts/test-results.txt` также сохранены сфокусированные команды:

```
./gradlew :core:jvmTest --tests persistence.RoomPersistenceTest
./gradlew :core:jvmTest --tests persistence.RoomInvariantTest
./gradlew :core:jvmTest --tests application.ControlScenarioTest
./gradlew :app:jvmTest --tests ui.ViewModelValidationTest
./gradlew :app:jvmTest --tests ui.ViewModelOperationsTest
```

Тест RoomPersistenceTest использует временный файл SQLite, закрывает первую базу, проверяет наличие файла, открывает базу снова и сравнивает восстановленные доменные объекты. Тесты инвариантов подтверждают защиту связей, возврат ровно один раз и откат неуспешного seed. RoomAdministrationTest проверяет domain-facing границу и транзакционную очистку, а ViewModelOperationsTest – фильтрацию, удаление и передачу ошибок в UI. ManyToManyTest оформляет две выдачи одной книги разным читателям.

В логе сборки дополнительно отмечено предупреждение текущего JDK о native access для Bundled SQLite. Оно не является ошибкой и не изменило результат BUILD SUCCESSFUL. Финальное окно было запущено через :app:run и визуально проверено; снимки Readers, Loans и Architecture приведены в разделе 3.1.

ЗАКЛЮЧЕНИЕ

В курсовом проекте разработано настольное JVM-приложение для учёта читателей, публикаций, филиалов и выдач. Цель достигнута: предметная область представлена иерархией LibraryEntity, Person → Reader/Librarian и Publication → Book/Periodical; Loan и LibraryBranch дополняют модель.

Связь Reader ↔ Publication реализована через Loan, что позволяет хранить историю операций и поддерживать множество выдач с каждой стороны. Технические идентификаторы генерируются как UUID и не являются частью пользовательского ввода. Room 3 и SQLite обеспечивают восстановление данных, а внешние ключи и транзакции защищают целостность. Выдача, возврат и демонстрационный seed согласованы с инвентарём, причём повторный возврат и повторная загрузка данных идемпотентны.

Семь паттернов имеют конкретные точки применения: Repository скрывает Room, MVVM разделяет экран и состояние, Strategy инкапсулирует LoanPolicy, Factory создаёт валидные объекты, Observer передаёт изменения от Flow к StateFlow, State описывает жизненный цикл Loan, а Facade предоставляет сценарии LibraryService.

Фактический прогон `./gradlew clean test build --no-daemon` завершился успешно: 17 тестов, 0 failures, 0 errors и 0 skipped. Финальный интерфейс поддерживает человекочитаемый поиск, создание, защищённое удаление, выдачу/возврат, явный repeat-safe seed и транзакционную очистку. Дальнейшее развитие может включать редактирование существующих карточек и миграции схемы без изменения выделенных границ слоёв.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Шумова Е. О. Объектно-ориентированное программирование: методические указания к выполнению курсового проекта. Санкт-Петербург: СПбГУАП, 2026.
2. `README.md` проекта Course OOP Library: описание финальной архитектуры, Room 3, модулей и команд проверки.
3. `context/TASK.md`, `context/context.yaml`, `context/requirements-checklist.md`, `context/TARGET_ARCHITECTURE.md` – исходные требования и целевая архитектура проекта.
4. Kotlin documentation [Электронный ресурс]. URL: https://kotlinlang.org/docs/home.html (дата обращения: 01.09.2026).
5. Compose Multiplatform documentation [Электронный ресурс]. URL: https://www.jetbrains.com/compose-multiplatform/ (дата обращения: 01.09.2026).
6. AndroidX Room documentation [Электронный ресурс]. URL: https://developer.android.com/training/data-storage/room (дата обращения: 01.09.2026).
7. Gamma E., Helm R., Johnson R., Vlissides J. Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley, 1994.
8. Исходный код и тесты текущего проекта: каталоги `core/src`, `app/src`, `core/schemas`, `docs/images`.
9. Сохранённые результаты: `artifacts/test-results.txt`, `artifacts/build-results.txt`, `artifacts/tdd-room-rewrite.md`, `artifacts/code-results.md`.

ПРИЛОЖЕНИЕ А. РЕПРЕЗЕНТАТИВНЫЕ ФРАГМЕНТЫ CORE

В приложение включены небольшие фрагменты, достаточные для демонстрации контрактов, состояния и транзакционной границы. Полные исходные тексты остаются в каталогах проекта.

А.1. Контракт сущности и генератор UUID

Источник: `core/src/commonMain/kotlin/domain/LibraryEntity.kt`.

```
interface LibraryEntity {
    val id: String
}

interface IdGenerator {
    fun nextId(): String
}

class UuidIdGenerator : IdGenerator {
    override fun nextId(): String = Uuid.random().toString()
}
```

А.2. Состояние выдачи

Источник: `core/src/commonMain/kotlin/domain/DomainModels.kt`.

```
sealed interface LoanState {
    val status: String

    data object Active : LoanState {
        override val status: String = "ACTIVE"
    }

    data object Overdue : LoanState {
        override val status: String = "OVERDUE"
    }

    data class Returned(val returnedAt: Long) : LoanState {
        override val status: String = "RETURNED"
    }
}
```

A.3. Политика выдачи

Источник: `core/src/commonMain/kotlin/domain/LoanPolicy.kt`.

```
interface LoanPolicy {
    fun canIssue(reader: Reader, publication: Publication, activeLoansForReader:
Int): PolicyDecision
    fun dueAt(issuedAt: Long, publication: Publication): Long
}

class DefaultLoanPolicy(
    private val maxActiveLoansPerReader: Int = 5,
    private val bookLoanDays: Long = 21,
    private val periodicalLoanDays: Long = 7,
) : LoanPolicy {
    override fun dueAt(issuedAt: Long, publication: Publication): Long {
        val days = if (publication is Periodical) periodicalLoanDays else
bookLoanDays
        return issuedAt + days * MILLIS_PER_DAY
    }
}
```

A.4. Транзакции DAO

Источник: `core/src/commonMain/kotlin/persistence/LibraryDao.kt`.

```
@Transaction
suspend fun returnLoanAtomically(loanId: String, returnedAt: Long): LoanRecord? {
    val current = getLoan(loanId) ?: return null
    if (current.returnedAt != null) return current
    if (markReturned(loanId, returnedAt) != 1) return getLoan(loanId)
    if (incrementAvailable(current.publicationId) != 1) {
        error("Publication inventory invariant violated")
    }
    return getLoan(loanId)
}
```

ПРИЛОЖЕНИЕ Б. РЕПРЕЗЕНТАТИВНЫЕ ФРАГМЕНТЫ APP И ТЕСТОВ

Б.1. JVM-точка входа

Источник: app/src/jvmMain/kotlin/main.kt.

```
fun main() {
    val container = ApplicationContainer()
    runBlocking { container.service.seedDemo() }
    application {
        Window(
            onCloseRequest = {
                container.close()
                exitApplication()
            },
            title = "Course OOP Library",
        ) {
            App(container)
        }
    }
}
```

Б.2. Наблюдаемое состояние ViewModel

Источник: app/src/commonMain/kotlin/ui/LibraryViewModels.kt.

```
val state: StateFlow<CollectionUiState<Reader>> = repository.readers
    .map { CollectionUiState(items = it, isLoading = false) }
    .catch { emit(CollectionUiState(isLoading = false, error = it.message ?: "Unable
to load readers")) }
    .combine(errors) { value, error → value.copy(error = error ?: value.error) }
    .stateIn(scope, SharingStarted.Eagerly, CollectionUiState())
```

Б.3. Проверка персистентности и идемпотентного возврата

Источник: core/src/jvmTest/kotlin/persistence/RoomPersistenceTest.kt.

```
val returned = service.returnLoan(loan.id, 2_000L)
val repeated = service.returnLoan(loan.id, 3_000L)
assertEquals(returned, repeated)
assertEquals(1, repositories.publications.findById(book.id)!!.availableCopies)
```

Б.4. Проверка M:N-связи

Источник: `core/src/jvmTest/kotlin/persistence/ManyToManyTest.kt`.

```
service.issueLoan(firstReader.id, book.id, librarian.id, 1L)
service.issueLoan(secondReader.id, book.id, librarian.id, 2L)

assertEquals(2, repositories.loans.loans.first().size)
assertEquals(0, repositories.publications.findById(book.id)!!.availableCopies)
```

Б.5. Идемпотентный seed

Источник: `core/src/jvmTest/kotlin/application/ControlScenarioTest.kt`.

```
assertTrue(service.seedDemo())
assertFalse(service.seedDemo())
assertEquals(2, repositories.readers.readersOnce().size)
assertEquals(2, repositories.publications.publicationsOnce().size)
assertEquals(1, repositories.loans.loans.first().size)
```

Эти фрагменты являются выборкой из финальных исходников и тестов; они не заменяют файлы проекта.